AI usage and Disclosure

GAI Tool Used: ChatGPT
GAI Version: GPT-6 Luna

Date Consulted: 09/27/2026

Exact Prompt:

"Problem Statement: Please write a program in C with the goal that it is able to take in a line of text and extract just a single IPv4 address and a port number if provided from anywhere in the provided text. The only valid tokens are digits (0-9), colons (:), and periods (.) and other other characters should be skipped. The entire piece of text is required to fit these address rules from start to finish, noting that even one character failing to fit the structure or the text containing additional characters surrounding a valid address, it will be rejected in its entirety (no partial matches, no truncating to find a valid piece inside a longer run). An address is identified as four octets separated by periods using the following format, octet.octet.octet.octet
Note that each octet can contain 1-3 digits ranging from values of 0-255. Leading zeros are not allowed unless the value itself is exactly 0. As mentioned above, a port following the address fourth octet is allowed with the specifications on that being it can contain 1-5 digits with values ranging from 0–65535. Again leading zeros are not permitted unless the value itself is exactly 0. If a colon is present the port is required to be fully valid; if it does not meet these requirements then the entire match, including the address, is rejected.

Function Prototype: The function returns whether a valid address was found, and delivers the address and port through two additional parameters. Behavior specifications have been provided below for your reference.
C:
// Returns 1 if a valid address was found, 0 otherwise.
// On success: *outAddress holds the 32-bit value, and
// *outPort holds the port number, or -1 if no port was present.
// On failure: *outAddress is set to 0 and *outPort is set to -1.
int extractIPv4(const char* str, unsigned long* outAddress, int* outPort);

Program Requirements: You may not use any of the following built-in functions or libraries:

- Any string-to-number conversion function: atoi, atol, atoll, strtol, strtoul, strtod, stoi, stol, stoul, sscanf, scanf with numeric conversions.
- Any address-parsing library function: inet_aton, inet_pton, inet_addr, or equivalents.

- Any regular-expression facility (std::regex, POSIX regex.h, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library.
- Standard character-classification functions (isdigit, etc.) are fine to use.

I have included/elaborated on other program requirements below:

- Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.
- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.
- On success, print: Extracted IPv4 address: A.B.C.D (decimal value: N, port: P) where N is the 32-bit decimal value and P is the port number or the literal text none.
- Input loop: main continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits.
- Extraction function: implement extractIPv4 exactly as prototyped above. All digit accumulation must be done by hand.
- Display: main receives the result from extractIPv4 and formats the output exactly as specified above.

Example of a successful run: Below is an example of a successful program run, which you should use to format your code after.

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port: 8080)
Enter a string (or 'END' to quit): 192a168.1.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port: none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port: none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
```

Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated."

Secondary Prompt upon test case failure:
"connecting to 192.168.1.1 then 192.168.10.10:302
//this test case does not return a valid address but it should return the first"

Code Attribution:

All parts of the submitted code are AI generated and student verified. I modified the extractIPV4 function within the AI generated code, due to it not passing the test case with two valid addresses. I told the GAI model to rewrite the function so that in the test case of "connecting to 192.168.1.1 then 192.168.10.10:302" it would output the first address instead of outputting that there was no valid address.

Verification Statement:

I confirm that I understand every line of the submitted code and have tested it using 25 test cases and confirmed it works as intended meeting the assignment requirements. There are no known bugs, limitations, or unexpected behavior in the submitted code.